#13 캠 가동 — 1분 큰 그림 + 셀프 점검

출처: Notion `2차프로젝트 → [#13] 코드리뷰 · 캠 가동` (PR #112)

1분 큰 그림 (중학생 버전)

상황: 등록만 한 카메라는 **꺼진(정지) 상태**다. 관리자가 "가동" 버튼을 눌러야 분석을 시작한다.

문제: 카메라 영상을 읽고 사람 수를 세는 일은 **분석 서버**(다른 프로그램)가 한다. 우리 관리자 앱은 분석 서버에게 "이 카메라 분석 시작해"라고 **전화로 시켜야** 한다. 그런데 전화가 안 걸릴 수도 있다.

해결 (#13): 가동 버튼을 누르면 ① 카메라가 켤 수 있는 상태인지 확인하고 ② "켜야 한다"는 메모를 남기고 ③ 분석 서버에 시작을 지시하고 ④ 돌아온 답으로 카메라 상태를 바꾸는 API를 만들었다. **실패해도 ERROR 상태와 "켜야 한다"는 메모는 남긴다.**

비유 (교실 스위치 켜기): 스위치를 누르기 전에 "지금 꺼져 있나?" 확인하고, "켜야 한다"는 메모(`run_requested`)를 붙여 둔 뒤 관리실(분석 서버)에 전화한다. 전화가 안 되면 메모는 남겨 두어, 관리실이 다시 켜지면 알아서 켜진다.

서버가 하는 일 4단계

1. 카메라가 있고 상태가 **STOPPED 또는 ERROR**인지 확인한다. 아니면 409 `INVALID_STATUS`. 통과하면 `run_requested = 1` 메모를 남긴다.
2. 분석 서버에 `start`를 호출한다. (**5초** 안에 답이 없으면 포기)
3. **성공**이면 응답의 상태(PREPARING 또는 RUNNING)로 카메라 상태를 바꾼다. 처음 켜진 시각(`first_started_at`)이 비어 있으면 지금 시각을 넣는다.
4. **실패**면 카메라를 ERROR로 두고 오류 내용을 남긴다.

상황	응답	카메라 상태
성공	200 `{status: PREPARING}`	PREPARING 또는 RUNNING
켤 수 없는 상태(이미 켜짐 등)	409 `INVALID_STATUS`	그대로
없는/삭제된 카메라	404	-
분석 서버 연결 실패·5초 초과·그 밖의 오류	502 `ANALYZER_UNAVAILABLE`	ERROR ('분석 서버 연결 실패')
분석 서버가 "영상 파일 없음"이라고 답함	422 `SOURCE_NOT_FOUND`	ERROR ('영상 파일·스트림 없음')

두 실패 경우 모두 `run_requested = 1`은 **유지**한다.

이 PR에서 제일 중요한 개념: 트랜잭션을 세 단계로 쪼갠 이유

#7에서 배운 `@Transactional`은 "전부 되거나 전부 취소"였다. 그런데 **여기서는 일부러 `@Transactional`을 뺐다.** 왜일까?

만약 메서드 전체를 한 묶음으로 만들면 문제가 2개 생긴다.

문제 ① DB 연결을 5초 동안 붙들고 있는다.

분석 서버가 5초 걸려 답하는 동안, 우리는 DB 연결을 잡고 아무것도 안 하고 기다린다. 도서관에서 책 한 권을 빌려 놓고 자리를 5초 동안 비우지 않는 것과 비슷하다. 다른 사람이 쓰지 못한다.

문제 ② 실패 상태가 같이 취소된다. (더 치명적)

분석 서버 연결이 실패하면 우리는 카메라를 ERROR로 바꾸고 예외를 던진다. 그런데 한 묶음이면 **예외가 나는 순간 "ERROR로 바꾼 것"까지 같이 취소(롤백)된다.** 명세는 "실패해도 ERROR와 run_requested=1을 남겨라"고 했으니 이러면 명세를 못 지킨다.

그래서 세 번 따로 커밋한다.

```
① 확인 + run_requested=1 기록   → 여기서 커밋 (확정)
② 분석 서버에 전화 (최대 5초)    → DB 묶음 밖
③ 결과 기록 (상태 갱신/ERROR)    → 여기서 커밋 (확정)
```

이러면 ②가 실패해도 ①은 이미 확정됐고, ③에서 ERROR를 기록하고 나서 예외를 던져도 ③은 이미 확정됐다. 이 구조를 코드의 `TransactionTemplate`이 만든다.

왜 `run_requested = 1`을 전화보다 먼저 확정하나? 전화가 실패하거나 앱이 중간에 죽어도 "켜야 한다"는 기록이 남아 있어야, 분석 서버가 나중에 다시 켜졌을 때 그 기록을 보고 **자동으로 가동**하기 때문이다.

핵심 개념 3가지 더

① 분석 서버를 부르는 코드는 `AnalyzerClient` 한 곳에 모은다

service는 "판단"만 하고, 다른 서버를 부르는 일은 `client` 패키지에만 둔다. 정지(#14)도 같은 클라이언트를 쓴다.

② `X-Internal-Token` 헤더는 비밀 확인표다

관리자 앱 ↔ 분석 서버 요청에 붙이는 비밀 표시다. 값은 코드에 쓰지 않고 환경변수 `INTERNAL_API_TOKEN`에 넣는다. 설정 이름은 팀원 #15 브랜치와 같게 맞췄다.

③ 응답이 먼저 온 상태 보고를 덮어쓰지 않는다

분석 서버가 "시작했다"는 답과 별개로 상태 보고(#15)를 먼저 보낼 수 있다. 이미 RUNNING으로 바뀌었는데 우리가 PREPARING으로 덮어쓰면 틀린 상태가 된다. 그래서 3단계에서 "아직 STOPPED/ERROR일 때만 상태를 바꾼다"로 막았다. (`first_started_at`은 그래도 채운다)

반드시 알아야 할 것 두 가지

1) 작업 기록(처리 순서 5)이 빠졌다. #6이 아직 없다. 또 #6이 들어오면 세 단계를 따로 커밋하는 구조라서 "기록 저장이 실패하면 가동 결과를 어떻게 할지"를 새로 정해야 한다. (R1)

2) 실제 분석 서버와 연결하지 못했다. 테스트는 같은 JVM에 띄운 **가짜 HTTP 서버**로 했다. 이슈 완료 기준 "PREPARING → RUNNING 전환 확인" 중 이 PR은 "응답이 PREPARING 또는 RUNNING이면 그 상태로 바뀐다"까지만 확인한다. PREPARING 뒤 RUNNING이 되는 건 분석 서버의 상태 보고(FNC-A-025, 팀원 #15)가 있어야 한다. (R2)

솔직하게 말해야 할 약점

1. **작업 기록이 없다.** #6 대기. (R1)
2. **실제 분석 서버와 연결 못 했다.** 가짜 서버로만 확인. (R2)
3. **권한 검사가 없다.** 보안 설정이 dev에 없다. (R3)
4. **명세가 정하지 않은 분석 서버 응답을 내가 정해서 처리했다.** 401(토큰 불일치), 501(LIVE 미지원), 400의 다른 코드, 5xx, 200인데 알 수 없는 상태 값, JSON이 아닌 본문을 모두 502 + ERROR('분석 서버 연결 실패')로 처리한다. 특히 **LIVE 캠을 가동하면 "분석 서버 연결 실패"라고 보여 오해를 줄 수 있다.** (R4)
5. **같은 캠에 가동 요청이 동시에 오면 분석 서버를 두 번 부른다.** 분석 서버가 멱등(두 번 불러도 같은 결과)이라 괜찮지만 잠금은 없다. (R5)
6. **요청 ID(X-Request-Id) 전달이 없다.** 공통 필터가 dev에 없다. (R6)
7. **1단계와 3단계 사이에 앱이 죽으면** 카메라는 STOPPED/ERROR인데 `run_requested=1`로 남는다. 의도한 동작이지만 화면에는 정지로 보일 수 있다. (R7)

셀프 점검 (자기 말로 답해 보기)

Q1. 이 API는 뭘 하는 건가요? 분석 서버는 뭘 하는 곳인가요?

답:

Q2. 왜 `run_requested = 1`을 분석 서버에 전화하기 **전에** 먼저 저장하나요?

답:

Q3. 이 메서드에는 왜 `@Transactional`을 안 붙이고 세 단계로 쪼갰나요? (문제 2가지)

답:

Q4. 분석 서버 연결이 실패하면 카메라 상태와 응답은 어떻게 되나요?

답:

내일 반드시 말할 수 있어야 하는 4가지

1. 이 API가 뭘 하고(분석 서버에 시작 지시) 왜 필요한가
2. 트랜잭션을 세 단계로 쪼갠 이유 (DB 안 붙들기 + 실패 상태 롤백 방지)
3. 실패해도 `ERROR`와 `run_requested=1`이 남는 이유 (분석 서버가 다시 켜지면 자동 가동)
4. 약점: 작업 기록 미구현, 실제 분석 서버 미연결, 명세에 없는 응답은 내가 정해 처리